# Cognitive Robotics: Comparative Evaluation of Hybrid LLM-Symbolic Instruction Pipelines

Peter Yaacoub

*Facultat d'Informàtica de Barcelona, Universitat Politècnica de Catalunya (UPC)*

**Course:** Introduction to Research, Bachelor Degree in Artificial Intelligence
**Professors:** Ramon Sangüesa Solé, Cecilio Angulo Bahon

May 2026

**Abstract**

This work evaluates four instruction-to-action pipelines for cognitive robotics under a shared OWL/RDF world model and a common natural-language benchmark: **Soar**, **ACT-R**, **Ontology Executor**, and a **Pure LLM Baseline**. The benchmark contains **53 commands** across five categories (Simple, Attribute-based, Spatial Relations, Ambiguous, Multi-step), executed for **3 seeded runs** per command and pipeline (636 total evaluations, 159 per pipeline). All pipelines consume the same JSON action schema produced by the same parser, then diverge at execution time.

Quantitatively, all pipelines reached **100%** action-sequence match to gold JSON and a **100%** task completion rate under the study's safe-rejection definition. Constraint violations were consistent across architectures: **45/159** runs per pipeline, i.e. 0.283, concentrated in under-specified or physically impossible instructions. The major separation appears in efficiency: Soar/ACT-R/Ontology required **13.04–13.15 s** mean planning latency and **1286** mean tokens, while the pure LLM baseline required **37.20 s** and **5579** tokens (∼65% slower and ∼77% more tokens).

These findings support a practical division of labor: LLMs are effective semantic translators, while symbolic/ontological layers provide grounded control and explicit constraint enforcement. The resulting hybrid pattern remains interpretable, safe, and substantially more computationally efficient than a fully neural planner in this benchmark.

The supporting scripts for this study are available in the project GitHub repository:
`https://github.com/Yaacoub/upc-fib-i2r-gia-cognitive-robotics/`

# Contents


**1 Introduction** **3**
- 1.1 State of the Art . . . 3
  - 1.1.1 Soar (State, Operator, And Result) . . . 3
  - 1.1.2 ACT-R (Adaptive Control of Thought-Rational) . . . 3
  - 1.1.3 Ontologies . . . 3
  - 1.1.4 Large Language Models (LLMs) . . . 4
- 1.2 Objectives . . . 4
  - 1.2.1 Specific Objectives . . . 4
  - 1.2.2 Key Performance Indicators (KPIs) . . . 5

**2 Project Management** **6**
- 2.1 Timeline . . . 6
- 2.2 Task Deliverables and Milestones . . . 6

**3 Methodology** **7**
- 3.1 Domain Modeling . . . 7
- 3.2 Dataset Creation . . . 7
- 3.3 Hybrid Integration . . . 11
  - 3.3.1 Shared Architecture . . . 11
  - 3.3.2 Explainability . . . 11
  - 3.3.3 Error Typology . . . 12
- 3.4 Pipeline A: Soar . . . 12
- 3.5 Pipeline B: ACT-R . . . 13
- 3.6 Pipeline C: Ontologies . . . 13
- 3.7 Pipeline D: LLM Baseline . . . 13

**4 Results** **14**
- 4.1 Quantitative Results . . . 14
- 4.2 Command-Level and Qualitative Findings . . . 14

**5 Discussion** **16**
- 5.1 Architectural Efficacy and Performance . . . 16
- 5.2 Cognitive and Theoretical Normalization . . . 16
- 5.3 Limitations and Assumptions . . . 16

**6 Conclusion and Future Work** **17**

# 1 Introduction

## 1.1 State of the Art

The interpretation of semantic instructions for robotic agents has evolved from symbolic reasoning within fixed architectures to dynamic pipelines leveraging foundation models. This section reviews the state of the art across four major paradigms.

### 1.1.1 Soar (State, Operator, And Result)

The **Soar** cognitive architecture provides a unified framework for achieving human-level intelligence in robotics, grounded in the **Problem Space Computational Model (PSCM)** where every goal-oriented task is formulated as a search through a problem space using **operators** to transform **states** [1,2]. At its core, the architecture operates via a repetitive **decision cycle** driven by parallel production rules that generate **preferences** for operator selection. Whenever these preferences are insufficient to determine a unique action, the system automatically initiates **universal subgoaling** to resolve the resulting **impasse** [1,2]. This deliberative process is refined through **chunking**, a pervasive learning mechanism that caches the results of substate reasoning into new procedural rules, effectively converting complex internal deliberation into reactive knowledge [1,3]. While early robotic implementations like **Robo-Soar** successfully integrated symbolic planning with real-time execution in manipulation tasks [3], modern developments in the current version, **Soar 9**, have transitioned the system into a hybrid architecture by incorporating **reinforcement learning**, **semantic memory**, and **episodic memory** to handle large-scale, knowledge-rich domains [2,4]. Most notably for robotic autonomy, Soar now utilizes a **Spatial Visual System (SVS)** that enables **mental imagery** and the transduction of continuous environmental data into symbolic relational structures, allowing agents to perform hypothetical reasoning on non-symbolic representations and learn continuous action models from experience [2,4].

### 1.1.2 ACT-R (Adaptive Control of Thought-Rational)

The **ACT-R** architecture facilitates the development of robotic agents capable of human-like cognitive fidelity by employing a **hybrid cognitive framework** that integrates symbolic **chunks** and **production rules** with subsymbolic **activation** and **utility** mechanisms [5,6]. It is organized into specialized, independent **modules**, including visual, manual, goal, and declarative memory, that communicate through limited-capacity **buffers** synchronized by a central production system [5,7]. The architecture operates on a discrete cycle, typically estimated at 50 ms, where patterns in the buffers are recognized and acted upon to drive behavior in real-time [5]. In the robotics domain, the most significant modern advancement is the evolution of **ACT-R/E** (Embodied), which extends the canonical architecture with **configural** and **manipulative modules** for **spatial reasoning** and **perspective-taking** in three-dimensional environments [8,9]. Current state-of-the-art implementations prioritize the modular integration of these cognitive processes into robotic frameworks like **ROS**, enabling **Cognitive Robotics** to move from rigid algorithmic control toward adaptive, human-centric collaboration [10].

### 1.1.3 Ontologies

To facilitate robust reasoning and collaboration, robotics research has increasingly adopted **ontologies** as a means of providing a **formal and explicit specification of shared conceptualizations** within the domain [11]. These frameworks move beyond simple taxonomies by incorporating **classes**, **relations**, and **axioms** to enable **semantic interoperability** between heterogeneous robotic systems and human stakeholders [12,13]. A pivotal milestone in this area was the development of the **IEEE 1872-2015 standard**, which introduced the **Core Ontology for Robotics and Automation (CORA)** to unify fundamental concepts across industrial, service, and autonomous applications [14]. In practice, processing frameworks such as **KnowRob** implement these principles through logic-based architectures, successfully bridging **low-level sensory-motor data** with **high-level semantic knowledge** to support complex, action-centered manipulation tasks [15]. More recent advancements have transitioned toward **knowledge-enabled cloud systems**, such as **RoboEarth**, which empower robots to share **machine-understandable** experiential data and modular knowledge chunks across global networks [16]. Ultimately, this structured representation of the physical world provides the essential **symbolic grounding** and levels of abstraction necessary for robots to interpret the complex contextual cues that characterize modern language-driven interaction [17].

### 1.1.4 Large Language Models (LLMs)

The transition from traditional symbolic systems to **foundation models** has been catalyzed by the emergence of **LLMs**, which are fundamentally based on the **Transformer architecture** [18]. Utilizing **scaling laws** and **in-context learning**, these models encode vast amounts of semantic knowledge from Internet-scale text corpora [19]. Within the robotics domain, LLMs serve as high-level planners capable of **zero-shot generalization** to novel instructions and environments, effectively bridging the gap between natural language commands and actionable robot logic [20]. This paradigm has evolved into the automated generation of structured control representations, such as **behavior trees** (BTs), which allow LLMs to assemble complex, modular robotic logic that remains interpretable and structurally correct [21]. While these advancements enable the execution of intricate, **long-horizon tasks**, ensuring reliability remains an active area of research. Recent frameworks propose the use of **safety constraint modules** and formal logic verification to provide verifiable guarantees against prohibited actions [22]. However, as these purely text-based systems transition toward autonomous interaction, a critical challenge emerges: grounding these abstract linguistic plans into the physical affordances and visual realities of the natural world. Vision-Language Models (VLMs) address this grounding gap through multimodal architectures that align visual and textual representations [17, 23, 24], but since this study isolates text-to-action reasoning using equivalent symbolic environment representations, VLMs are outside the present scope.

## 1.2 Objectives

This study aims to perform a comparative analysis of robotic reasoning pipelines by framing the architecture through three distinct functional roles: purely probabilistic models acting as **interpreters** (LLMs), explicit knowledge representations functioning as **constraint systems** (Ontologies), and biologically-inspired symbolic architectures serving as **control systems** (Soar and ACT-R). While recent advancements have introduced Vision-Language Models (VLMs) for direct sensory grounding, this study isolates and evaluates the foundational **text-to-action reasoning capabilities** by providing all models with equivalent, text-based symbolic representations of the environment state.

### 1.2.1 Specific Objectives

1. **Environment Modeling**: Develop a formal environment state representation using Protégé and OWL as the universal ground truth queried by all four pipelines. Each pipeline receives the world state in its native format: Soar and ACT-R via translated Working Memory Elements and Declarative Chunks respectively, the Ontology pipeline by querying the `.owl` file directly, and the LLM baseline via a dynamically constructed textual prompt.

2. **Dataset Design**: Curate a standardized dataset of natural language instructions (**53 items**) structurally divided into five distinct categories. This taxonomy, detailed in Table 1, guarantees comprehensive coverage of interaction complexities and allows for fine-grained analysis of system performance across different levels of linguistic and spatial difficulty.

Table 1: Instruction Dataset Categorization

| Category | Description | Example |
|---|---|---|
| **Simple** | Single-action commands with direct, unambiguous parameters. | "Move north.", "Move to (2,2)." |
| **Attribute-based** | Commands requiring object identification via specific descriptive properties. | "Pick up the green apple." |
| **Spatial Relations** | Instructions involving relative positioning, inclusion, or orientation. | "Is the yellow cup on the table?" |
| **Ambiguous** | Commands lacking explicit parameters that require contextual inference (e.g., deixis). | "Drop the item.", "Head towards the trashcan." |
| **Multi-step** | Sequential tasks combined into a single continuous utterance. | "Pick up the green apple, then drop it." |

3. **Pipeline Implementation**: Construct and validate four distinct interpretation pipelines that translate natural language instructions into logical action sequences:

- Pipeline A: Symbolic goal-driven planning using the Soar architecture.
- Pipeline B: Memory-based retrieval and rule selection using ACT-R.
- Pipeline C: Direct knowledge-driven reasoning and instruction mapping using an Ontology.
- Pipeline D: Pure LLM-based reasoning (Zero-shot/Few-shot Baseline).

4. **Comparative Evaluation**: Benchmark the four pipelines against technical metrics using a light, headless symbolic state-tracker (rather than a 3D physics engine). This isolates and evaluates the text-to-logic reasoning capabilities independently of kinematic or hardware constraints.

### 1.2.2 Key Performance Indicators (KPIs)

To rigorously compare the architectures, five Key Performance Indicators (KPIs) were established, as defined in Table 2.

For KPI-01 (Interpretation Accuracy), the LLM's parsed output is evaluated against a manually defined gold standard. It is measured both via binary Exact Match and a granular Partial Correctness score (isolating Action, Object Identification, and Relation accuracy). For instance, the gold standard for the first command, "Move to (2,2)", must strictly align with this format:

```
[{"action":"move","desired-x":2,"desired-y":2}]
```

For the execution phases, KPI-02 (Task Success Rate) requires the final state to align with the goal, safely rejecting invalid commands. KPI-05 (Constraint Violation Rate) evaluates the structural contribution of the symbolic systems by explicitly tracking attempts to violate physical or semantic rules (e.g., out-of-bounds navigation or incorrect capacity assumptions).

Table 2: Project Key Performance Indicators

| KPI ID | Metric Name | Definition / Target |
|---|---|---|
| KPI-01 | Interpretation Accuracy | Evaluated against a predefined gold standard. Measured via Exact Match (binary) and Partial Correctness (Action, Object Identification, Relation). |
| KPI-02 | Task Success Rate | Percentage of tasks successfully completed in the simulated environment. |
| KPI-03 | Planning Latency | Total time (s) required to generate an executable plan, explicitly partitioned into neural generation time (LLM parsing) and symbolic validation/reasoning time. |
| KPI-04 | Model Complexity | LLM token usage (prompt, completion, and total) measured per command execution. |
| KPI-05 | Constraint Violation Rate | Total count of generated commands that violate predefined physical, spatial, or semantic domain boundaries. |

# 2 Project Management

The project is divided into three main phases, each concluding with technical milestones.

## 2.1 Timeline

To provide a clear estimate of the time required per task, Figure 1 illustrates the timeline of the project from its inception to its final delivery, based on an estimated workload of 180 hours.

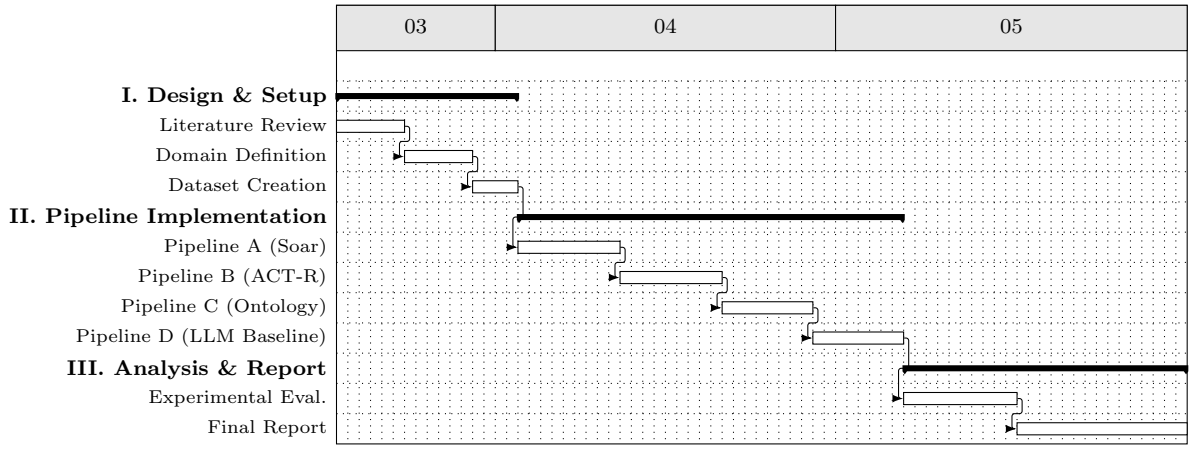


Figure 1: Project Estimated Time Planning (Gantt Chart)

In contrast, Figure 2 illustrates the project's effective timeline, based on the actual workload.

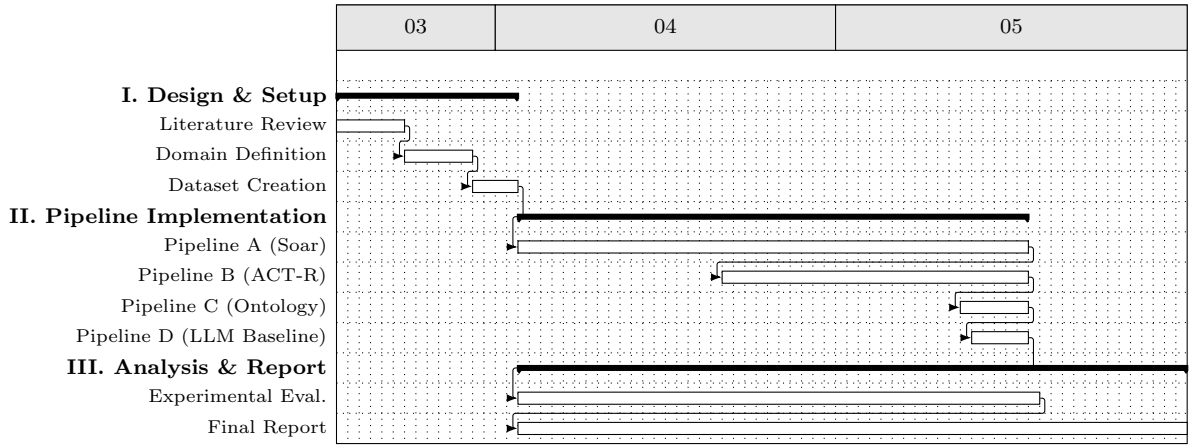


Figure 2: Project Effective Time Planning (Gantt Chart)

## 2.2 Task Deliverables and Milestones

The core deliverables yielded by each phase are as follows:

- **Phase I (Experimental Setup - 40h):** Contextualization of the state of the art, creation of the `.owl` domain file, and finalization of the standardized 53-command instruction set.
- **Phase II (Pipeline Implementation - 90h):** Development of functional prototypes for the four pipelines: Symbolic Soar, ACT-R, Ontology Reasoning, and the LLM Baseline.
- **Phase III (Analysis & Report - 50h):** Execution of the lightweight Python state-tracking script to gather Exact Match and Partial Correctness results, culminating in the analysis presented in this final paper.

# 3 Methodology

To ensure a rigorous and equitable comparative analysis across the four distinct processing pipelines, a shared, objective environment must be established. This section details the formal modeling of this environment, which serves as the universal ground truth state queried by the symbolic architectures, the ontological reasoner, and the LLM baseline.

## 3.1 Domain Modeling

The environment is formally represented as an OWL 2 (Web Ontology Language) ontology. To ensure interoperability and adherence to established robotics standards, the domain model is foundationally built upon the IEEE 1872-2015 Core Ontology for Robotics and Automation (CORA) [14]. Specifically, the `cora-bare` module is imported to provide the high-level definitions for robotic agents and environments.

Building upon this standard, a custom hierarchical taxonomy was designed to capture the specific physical entities required for the experimental manipulation and navigation tasks, as illustrated in Figure 3. The core hierarchy revolves around the `PhysicalObject` class, which is partitioned into two main sub-hierarchies:

- **Static Objects:** Environmental fixtures such as the `Table` and `TrashCan` that respectively provide a surface and a receptacle but cannot be displaced by the agent.
- **Manipulable Objects:** Items such as the `Apple` and `Cup` that the robot can dynamically grasp, transport, and eventually stack.

To facilitate attribute-based referencing in natural language (e.g., distinguishing between identical object classes), the datatype property `hasColor` was implemented to assign distinct visual properties to the manipulable instances. The robotic entity itself is modeled as an `Agent`, explicitly defined as a subclass of CORA's fundamental `Robot` class.

To support the spatial and relational reasoning essential for cognitive tasks, the ontology employs strict Description Logic (DL) semantics. Spatial grounding is achieved through the `GridLocation` class (a subclass of CORA's `RoboticEnvironment`), which utilizes data properties (`hasXCoordinate`, `hasYCoordinate`) to define a discrete coordinate system. In this specific experimental setup, the environment is constrained to a 4x4 grid spanning coordinates `[0,0]` to `[3,3]`, visually mapped in Figure 4.

The relationships between objects, agents, and locations are enforced through carefully constrained Object Properties. For instance, the `isAtLocation` property links both `PhysicalObject` and `Robot` entities to a `GridLocation` via a union domain constraint. Crucially, `isAtLocation` is declared as a **Functional Property**, enforcing the physical law that an entity can only occupy one exact coordinate at any given time. To facilitate bidirectional querying, allowing a model to natively interpret both "Where is the apple?" and "What is at location [2,2]?", inverse properties such as `containsObject` (inverse of `isAtLocation`) and `isHeldBy` (inverse of `isHolding`) were defined. This explicit semantic structure guarantees that all computational pipelines base their logic on identical, unambiguous environmental rules.

## 3.2 Dataset Creation

To ensure comprehensive coverage of the domain model, Table 3 maps each instruction against the specific ontological entities, properties, and operators it explicitly triggers or evaluates. Hence, the benchmark covers the reachable success, failure, and niche control paths in the current implementations.

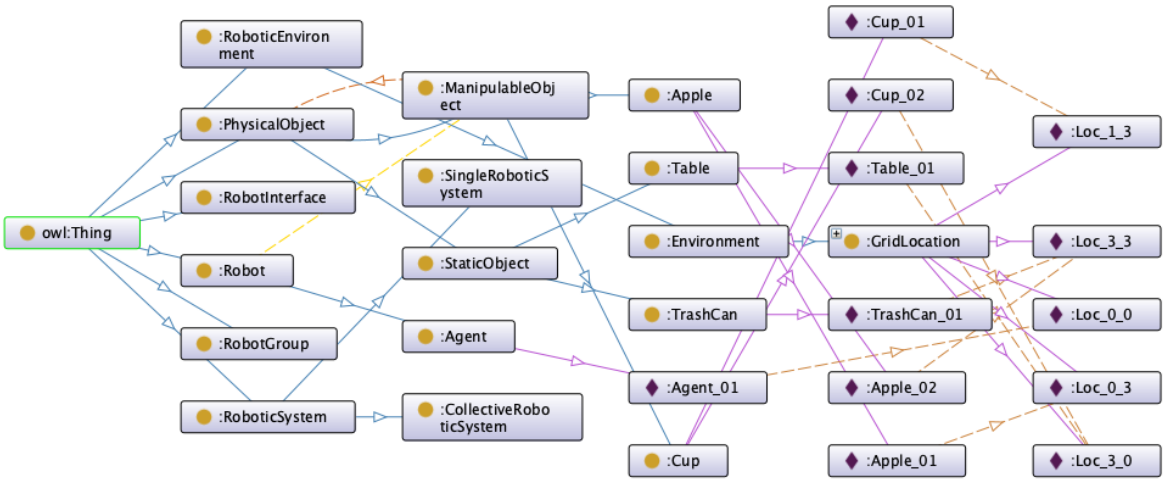


Figure 3: Visualization of the custom Cognitive Robotics Ontology, illustrating class hierarchies and spatial relations.

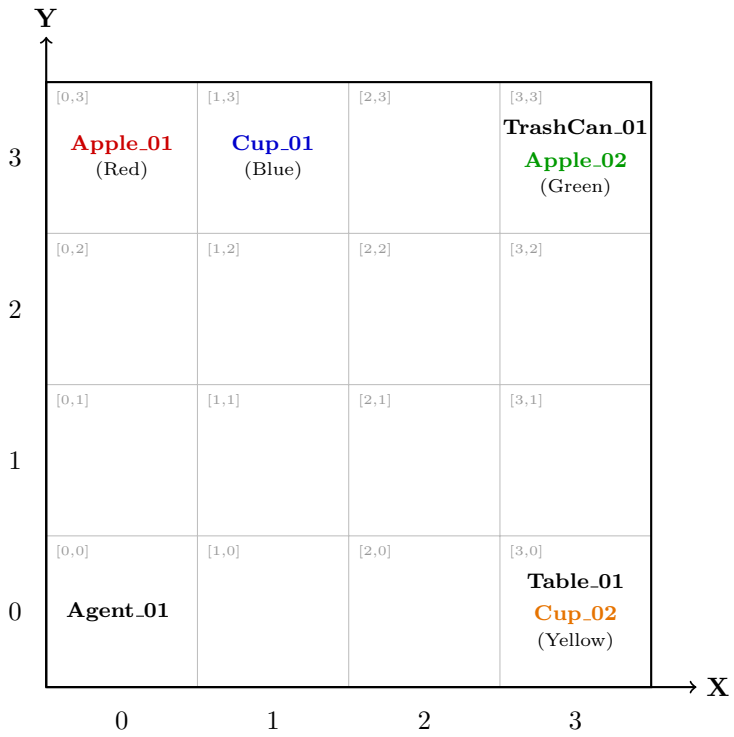


Figure 4: Spatial configuration of the initial environment state extracted from the OWL/RDF ontology, mapped to a 4x4 coordinate grid.

Table 3: Coverage-Oriented Semantic Command Dataset for Evaluation

| ID | Natural Language Instruction | Gold Action Schema (from `ground_truth.txt`) |
|---|---|---|
| **Simple** | | |
| **CMD-01** | Move to (2,2). | `move(desired-x=2;desired-y=2)` |
| **CMD-02** | Move to [-1, 5]. | `move(desired-x=-1;desired-y=5)` |
| **CMD-03** | Move north. | `move(direction=north)` |
| **CMD-04** | Move east. | `move(direction=east)` |
| **CMD-05** | Move south. | `move(direction=south)` |
| **CMD-06** | Move west. | `move(direction=west)` |
| **CMD-07** | Move up by 2. | `move(direction=north;distance=2)` |
| **CMD-08** | Move down by 3. | `move(direction=south;distance=3)` |
| **CMD-09** | Move forward 5 steps. | `move(distance=5)` |
| **Attribute-based** | | |
| **CMD-10** | Tell me the coordinates of the green apple. | `query-location(target-class=apple;target-modifiers=[green])` |
| **CMD-11** | Bring the red apple to loc_1_3. | `get(target-class=apple;target-modifiers=[red])->set(desired-x=1;desired-y=3)` |
| **CMD-12** | Bring the green apple to loc_0_0. | `get(target-class=apple;target-modifiers=[green])->set(desired-x=0;desired-y=0)` |
| **CMD-13** | Where is the yellow cup? | `query-location(target-class=cup;target-modifiers=[yellow])` |
| **CMD-14** | Where is the blue cup? | `query-location(target-class=cup;target-modifiers=[blue])` |
| **CMD-15** | Pick up the green apple. | `get(target-class=apple;target-modifiers=[green])` |
| **CMD-16** | Pick up the blue cup. | `get(target-class=cup;target-modifiers=[blue])` |
| **CMD-17** | Pick up the dirty green apple. | `get(target-class=apple;target-modifiers=[dirty,green])` |
| **CMD-18** | Where is cup_02 that is blue? | `query-location(target-class=cup;target-modifiers=[02,blue])` |
| **CMD-19** | Where is the apple? | `query-location(target-class=apple)` |
| **CMD-20** | Tell me the coordinates of the red apple. | `query-location(target-class=apple;target-modifiers=[red])` |
| **Spatial Relations** | | |
| **CMD-21** | Is the yellow cup on the table? | `query-boolean(target-class=cup;target-modifiers=[yellow];destination-class=table)` |
| **CMD-22** | Does the trashcan contain the green apple? | `query-boolean(target-class=apple;target-modifiers=[green];destination-class=trashcan)` |
| **CMD-23** | Does the trashcan contain the red apple? | `query-boolean(target-class=apple;target-modifiers=[red];destination-class=trashcan)` |
| **CMD-24** | Is the green apple on the table? | `query-boolean(target-class=apple;target-modifiers=[green];destination-class=table)` |
| **CMD-25** | Is the agent next to the table? | `query-boolean(target-class=agent;destination-class=table)` |
| **CMD-26** | Does the table hold the green apple? | `query-boolean(target-class=apple;target-modifiers=[green];destination-class=table)` |

| ID | Natural Language Instruction | Gold Action Schema (from `ground_truth.txt`) |
|---|---|---|
| **CMD-27** | Are you near the trashcan? | `query-boolean(target-class=agent;destination-class=trashcan)` |
| **CMD-28** | Is the yellow cup inside the trashcan? | `query-boolean(target-class=cup;target-modifiers=[yellow];destination-class=trashcan)` |
| **CMD-29** | Is the cup in the blue trashcan? | `query-boolean(target-class=cup;destination-class=trashcan;destination-modifiers=[blue])` |
| **Ambiguous** | | |
| **CMD-30** | Head towards the trashcan. | `move(destination-class=trashcan)` |
| **CMD-31** | Where is agent_01 located? | `query-location(target-class=agent;target-modifiers=[01])` |
| **CMD-32** | What are you holding? | `query-inventory(target-class=agent)` |
| **CMD-33** | Drop the item. | `set()` |
| **CMD-34** | Pick up the table. | `get(target-class=table)` |
| **CMD-35** | Go over there. | `move()` |
| **CMD-36** | Throw it away. | `set(destination-class=trashcan)` |
| **CMD-37** | Grab it. | `get()` |
| **CMD-38** | Put it down. | `set()` |
| **CMD-39** | Bring it to loc_4_4. | `set(desired-x=4;desired-y=4)` |
| **CMD-40** | Put that on the table. | `set(destination-class=table)` |
| **CMD-41** | Put the table inside the cup. | `get(target-class=table)->set(destination-class=cup)` |
| **CMD-42** | Hello robot, please be nice and pick up the green apple for me. Thanks! | `get(target-class=apple;target-modifiers=[green])` |
| **Multi-step** | | |
| **CMD-43** | Pick up the green apple, then tell me what you are holding. | `get(target-class=apple;target-modifiers=[green])->query-inventory(target-class=agent)` |
| **CMD-44** | Fetch the yellow cup and put it in the trashcan. | `get(target-class=cup;target-modifiers=[yellow])->set(destination-class=trashcan)` |
| **CMD-45** | Throw the red fruit into the bin. | `get(target-class=apple;target-modifiers=[red])->set(destination-class=trashcan)` |
| **CMD-46** | Put the red apple on the big table. | `get(target-class=apple;target-modifiers=[red])->set(destination-class=table;destination-modifiers=[big])` |
| **CMD-47** | Pick up the green apple, then pick up the yellow cup. | `get(target-class=apple;target-modifiers=[green])->get(target-class=cup;target-modifiers=[yellow])` |
| **CMD-48** | Pick up the green apple, then drop it. | `get(target-class=apple;target-modifiers=[green])->set()` |
| **CMD-49** | Move north, then pick up the blue cup. | `move(direction=north)->get(target-class=cup;target-modifiers=[blue])` |
| **CMD-50** | Get the red apple and drop it on the table. | `get(target-class=apple;target-modifiers=[red])->set(destination-class=table)` |
| **CMD-51** | Bring the yellow cup to loc_2_2 and then move south. | `get(target-class=cup;target-modifiers=[yellow])->set(desired-x=2;desired-y=2)->move(direction=south)` |
| **CMD-52** | Check if the yellow cup is on the table, then move east. | `query-boolean(target-class=cup;target-modifiers=[yellow];destination-class=table)->move(direction=east)` |
| **CMD-53** | Pick up the blue cup, drop it in the trashcan, and move west. | `get(target-class=cup;target-modifiers=[blue])->set(destination-class=trashcan)->move(direction=west)` |

## 3.3 Hybrid Integration

This research explicitly decouples natural language interpretation from physical execution: neural LLMs act purely as probabilistic semantic interpreters, Ontologies provide declarative physical constraints, and Cognitive Architectures (Soar & ACT-R) serve as the deterministic control engines executing the structured JSON logic.

### 3.3.1 Shared Architecture

All four computational pipelines are built upon a shared architectural framework that standardizes parsing, orchestration, and evaluation logic. A lightweight Python middleware decouples natural language interpretation from physical grounding, allowing the individual symbolic systems to operate solely as execution engines.

- **Dataset Loader:** Systematically reads the 53 paired command/gold benchmark entries via `load_dataset.py`.
- **Semantic Parser:** A 26-billion parameter LLM (`a4b` quantized for low-latency edge execution) serves as a universal semantic translator, ensuring robust zero-shot JSON schema adherence. To balance stochasticity and reproducibility, the model is queried at temperature 1.0 using deterministic seeds. Outputs are cached so all pipelines evaluate identical prompts.
- **Testing Harness & KPI Computation:** A centralized driver orchestrates the execution pipelines. Per-command results, including execution traces and token usage, are encapsulated in a `PipelineResult` dataclass. The harness invokes shared functions to generate `KPIScore` evaluations, saving the outputs to `Pipelines/Tests/outputs/[Pipeline]/CMD-XX-RYY.json` to facilitate cross-run aggregation.
- **Execution Tracing & Traceability:** To enable direct behavioral comparisons, all runtimes emit execution events normalized into a standard format. Trace markers include `[ACTION]` (physical state transitions like move, get, set), `[QUERY]` (epistemic outputs), `[SUCCESS]` (completed command sequences), `[FAILURE]` (symbolic constraint rejection), and `[STUCK]` (no-terminal-condition edge cases).

Because all execution pipelines leverage this shared state and parsing logic, the system universally inherits the following validated properties:

- **Constraint-First Safety:** Rather than relying on the LLM for physical compliance, each pipeline's core symbolic technology actively detects and blocks constraint violations prior to execution. This includes rejecting out-of-bounds coordinates, manipulation of static objects, capacity limits, and empty-gripper `set` commands.
- **Grounded Deixis:** Ambiguous commands (e.g., "drop it" or "grab it") are resolved by reading the agent's current simulated holding state, preventing LLM hallucination or guessing.
- **State-Aware Querying:** Query operators strictly read the current symbolic world state to produce explicit boolean, spatial, or inventory responses.
- **Deterministic Termination:** Command chains are guaranteed to terminate through explicit success, failure, or stuck logic.

### 3.3.2 Explainability

Within the **LLM** semantic parser, explainability is not measured through opaque internal weights, but through explicit structural constraints. To serve as a measurable proxy for interpretability, the prompt engineering completely restricts the LLM to outputting a predefined JSON schema corresponding to actionable cognitive commands. Any misinterpretation of natural language, such as hallucinated objects or incorrect spatial routing, is immediately isolated within discrete key-value pairs (e.g., `target-class`, `desired-x`, or `destination-modifiers`) prior to physical execution. This structured mapping, combined with input-output token usage tracking, provides a clear, qualitative rubric for evaluating reliability.

In cognitive architectures like **Soar**, native debug logs evaluate the system at the micro-cognitive level, tracking every preference generation, operator application, and architectural impasse. Because the

Soar architecture loops continuously until a state no-change or a predefined step-limit is triggered, the execution trace is intentionally augmented with the aforementioned standardized trace markers. This maps internal operator sequencing directly to real-world behavioral outcomes, allowing researchers to pinpoint the exact cognitive cycle where a command succeeded or was rejected.

Similarly, the **ACT-R** pipeline establishes explainability by making its modular cognitive states structurally transparent. The trace structure tracks the instantiation of declarative chunks within the goal buffer and the subsequent firing of production rules. Transparency is achieved by observing the sequential activation of these modules, enabling a clear map between symbolic mechanisms and the resulting robotic actions. By generating explicit logs of buffer transitions, the system provides a traceable proxy of how architectural constraints, such as manipulability, capacity limits, and perspective-taking, are actively enforced during execution.

### 3.3.3 Error Typology

To ensure a meaningful interpretation of system failures across the defined KPIs, any errors encountered during the text-to-action translation and symbolic execution phases are classified into a formalized taxonomy. This taxonomy distinctly separates failures of the **interpreter** from failures triggered by the **constraint system**:

- **Linguistic Parsing Errors:** Failures occurring when the foundation model incorrectly maps natural language syntax to the intended JSON action schema. This includes omitted parameters, misidentified target classes, or a failure to capture explicitly stated object modifiers (e.g., ignoring "green" when instructed to fetch a "green apple").
- **Semantic Errors:** Instances where the generated output is syntactically valid but semantically flawed given the predefined domain mapping rules. Examples include incorrectly generating a `set` action when a `get` (pick up) action is implicitly required, or assigning physical destination parameters to an incompatible query class.
- **Planning Errors:** Logical sequencing failures predominantly observed in multi-step commands. While individual atomic actions may be parsed correctly, the sequence is either out of order or misses critical intermediate prerequisites (e.g., failing to generate a `get` action before attempting to `set` a requested object into a bin).
- **World Knowledge Errors:** Failures occurring during the symbolic execution phase rather than the linguistic parsing phase. In these cases, the command sequence is linguistically and semantically sound but violates the physical or state constraints of the simulated environment (e.g., navigating out of bounds, attempting to manipulate an unreachably distant object, or exceeding inventory capacities).

This structured classification is essential for diagnosing the specific limitations of the LLM-to-symbolic pipeline. It directly informs the granular evaluation of Interpretation Accuracy (KPI-01) by isolating text-level failures, while explaining task failures captured in the Constraint Violation Rates (KPI-05) via world-knowledge grounding.

## 3.4 Pipeline A: Soar

The Soar pipeline translates the shared JSON action schema into grounded cognitive commands. By utilizing Soar as the deterministic control system, the LLM is explicitly forbidden from producing concrete instance IDs or resolving pronouns, those tasks are delegated entirely to Soar's symbolic logic.

The Soar instantiation employs the following specialized mechanisms:

- **Initialization Protocol:** The OWL/RDF world model is translated into initialization artifacts for the cognitive runtime. The environment is initialized through an explicit `init-environment` operator so that Working Memory Elements (WMEs) are operator-supported and can be safely modified by subsequent operators. Early elaboration-only patterns produced brittle behavior. Operator-supported initialization ensures robust state updates.
- **Knowledge Modules:** The cognitive engine is organized as modular `.soar` files implementing elaborations, proposals, applications, and safety checks. Constraint logic is enforced via elaboration-generated failure WMEs and reject rules that remove unsafe operator proposals from the decision cycle.

## 3.5 Pipeline B: ACT-R

This study implements a lightweight, functional Python abstraction instead of the official Lisp-based ACT-R 7 environment. This architectural adaptation was deliberately chosen to bypass the prohibitive middleware overhead of integrating legacy Lisp kernels with modern LLM APIs and JSON state trackers. Crucially, it strictly preserves ACT-R's core cognitive bottlenecks, limited-capacity buffers and declarative memory retrieval, allowing for robust evaluation of symbolic constraint handling without unnecessary sub-symbolic mathematical overhead.

The functional ACT-R pipeline enforces cognitive control through the following simulated mechanisms:

- **Declarative Memory Translation:** Prior to execution, the OWL environment state is translated into an ACT-R-inspired declarative memory JSON schema. Entities and their properties are instantiated as discrete chunks (e.g., `isa apple`, `isatlocation loc_0_3`).
- **Heuristic Subsymbolic Retrieval:** Instead of calculating strict base-level learning and spreading activation equations, chunk retrieval is governed by a heuristic scoring function. This function mimics sub-symbolic activation by prioritizing semantic feature matching (e.g., checking slot values against requested modifiers like "green") and access recency/frequency. This allows the system to probabilistically resolve ambiguous, descriptive targets into explicit physical chunk IDs.
- **Buffer-Driven Productions:** The traditional Rete pattern-matching engine is simulated via discrete, hard-coded conditional phases operating on a fixed 50 ms cognitive cycle. The architecture utilizes four primary, single-chunk capacity buffers: **Goal** (tracking the cognitive phase), **Imaginal** (holding the parsed action schema), **Retrieval** (holding fetched declarative chunks), and **Manual** (staging executable physical commands).

Execution begins by priming the *Goal* buffer with an `encode` phase and injecting the parsed JSON action sequence into the *Imaginal* buffer. The system then strictly loops through sequential production phases, logging each 50 ms step. Because the system relies on limited-capacity buffers, actions cannot physically execute until necessary prerequisites are pulled from memory.

## 3.6 Pipeline C: Ontologies

The ontology pipeline executes the parsed JSON action sequence directly over the RDF state using an integrated `OntologyWorld` and `OntologyExecutor`. For each command, the runtime loads the OWL/RDF graph and builds type/superclass indices to ensure class-aware retrieval.

Entities are resolved through a deterministic scoring function over class names, descriptive modifiers, color/type slots, and current holding bias. Upon successful resolution, the system executes `move/get/set/query-*` actions with explicit state mutations (e.g., updating `isAtLocation` and `isHolding` relationships) and immediately emits the standardized execution trace. Execution is strictly command-sequential and halts immediately on the first failure. Because the ontology runtime directly mutates and queries RDF triples, each trace line is inherently grounded in an explicit graph operation rather than a latent model state.

## 3.7 Pipeline D: LLM Baseline

The pure LLM baseline is intentionally two-stage and fully neural at planning time.

- **Stage 1 (Parsing):** Converts natural language to constrained JSON actions using the shared parser.
- **Stage 2 (Planning/Execution Simulation):** A simulated executor receives an OWL-derived textual world description and the parsed JSON action sequence. It then generates an ordered trace conforming to the standardized `[ACTION]/[SUCCESS]/[FAILURE]` contract used by the symbolic runtimes.

The planner prompt encodes explicit `move/get/set/query` transition rules. Unlike the other pipelines, this baseline does not execute through an external engine or graph model. Instead, the LLM hallucinates the step-by-step execution and returns a trace list. Parses and planning token costs are computationally summed to capture the end-to-end neural reasoning overhead.

# 4 Results

## 4.1 Quantitative Results

Table 4: Overall Performance Across 159 Runs per Pipeline

| Pipeline | KPI-01 | KPI-02 | KPI-03 (s) | KPI-04 | KPI-05 | LLM Time (s) | Symbolic Time (s) |
|---|---|---|---|---|---|---|---|
| Soar | 1.00 | 1.00 | $13.15 \pm 9.77$ | $1286 \pm 464$ | 0.283 (45/159) | 13.04 | 0.1091 |
| ACT-R | 1.00 | 1.00 | $13.04 \pm 9.77$ | $1286 \pm 464$ | 0.283 (45/159) | 13.04 | 0.0002 |
| Ontology | 1.00 | 1.00 | $13.04 \pm 9.77$ | $1286 \pm 464$ | 0.283 (45/159) | 13.04 | 0.0008 |
| LLM | 1.00 | 1.00 | $37.20 \pm 20.06$ | $5579 \pm 938$ | 0.283 (45/159) | 37.20 | 0.0000 |

Table 4 shows identical correctness scores but clear efficiency separation. Relative to the pure LLM baseline, Soar/ACT-R/Ontology reduce latency by approximately 64.7–65.0% and token usage by approximately 76.9%. This disparity in computational overhead is further detailed across the five instruction categories in Table 5 for planning latency and Table 6 for token consumption and constraint violation rates. Notably, multi-step commands exact the highest temporal and token costs across all architectures, though the LLM baseline's performance degrades much more severely.

Table 5: Category-Level Means (KPI-03, seconds)

| Category | Soar | ACT-R | Ontology | LLM |
|---|---|---|---|---|
| Simple | 7.68 | 7.57 | 7.57 | 20.01 |
| Attribute-based | 11.32 | 11.22 | 11.22 | 35.40 |
| Spatial Relations | 11.49 | 11.38 | 11.38 | 38.41 |
| Ambiguous | 15.34 | 15.23 | 15.23 | 34.59 |
| Multi-step | 18.21 | 18.10 | 18.10 | 55.17 |

Table 6: Category-Level Means (KPI-04 tokens and KPI-05 violation rate)

| Category | KPI-04 Soar/ACT-R/Ontology | KPI-04 LLM | KPI-05 (all pipelines) | Violating Commands |
|---|---|---|---|---|
| Simple | 1023 | 4789 | 0.556 | 5/9 |
| Attribute-based | 1210 | 5513 | 0.000 | 0/11 |
| Spatial Relations | 1209 | 5530 | 0.000 | 0/9 |
| Ambiguous | 1384 | 5456 | 0.692 | 9/13 |
| Multi-step | 1525 | 6477 | 0.091 | 1/11 |

## 4.2 Command-Level and Qualitative Findings

Table 7 details all 15 violating command templates using the error taxonomy defined in Section 3.3.3. The sole cross-pipeline divergence is CMD-37 (*"Grab it."*): Soar surfaces `unknown-target` while the other pipelines report `semantic-error`, an implementation-level labeling difference rather than a behavioral one. The remaining 38/53 commands produced `KPI-05 = 0` across all runs and pipelines.

Table 7: All Commands with Non-Zero KPI-05 (15/53 templates)

| ID | Category | Instruction | Dominant Error Type (Soar / ACT-R / Ontology / LLM) |
|---|---|---|---|
| CMD-02 | Simple | Move to [-1, 5]. | world-knowledge-error (All Pipelines) |
| CMD-05 | Simple | Move south. | world-knowledge-error (All Pipelines) |
| CMD-06 | Simple | Move west. | world-knowledge-error (All Pipelines) |
| CMD-08 | Simple | Move down by 3. | world-knowledge-error (All Pipelines) |
| CMD-09 | Simple | Move forward 5 steps. | world-knowledge-error (All Pipelines) |
| CMD-33 | Ambiguous | Drop the item. | planning-error (All Pipelines) |
| CMD-34 | Ambiguous | Pick up the table. | world-knowledge-error (All Pipelines) |
| CMD-35 | Ambiguous | Go over there. | world-knowledge-error (All Pipelines) |
| CMD-36 | Ambiguous | Throw it away. | planning-error (All Pipelines) |
| CMD-37 | Ambiguous | Grab it. | unknown-target (Soar) / semantic-error (Others) |
| CMD-38 | Ambiguous | Put it down. | planning-error (All Pipelines) |
| CMD-39 | Ambiguous | Bring it to loc_4_4. | planning-error (All Pipelines) |
| CMD-40 | Ambiguous | Put that on the table. | planning-error (All Pipelines) |
| CMD-41 | Ambiguous | Put the table inside the cup. | world-knowledge-error (All Pipelines) |
| CMD-47 | Multi-step | Pick up the green apple, then pick up the yellow cup. | world-knowledge-error (All Pipelines) |

# 5 Discussion

## 5.1 Architectural Efficacy and Performance

Because all four pipelines consumed a shared JSON action schema, KPI-01 and KPI-02 could not differentiate the architectures: the parser-produced sequences are identical inputs, and safe symbolic rejection of impossible commands counts as successful execution under our criteria. The core empirical differentiator is therefore **efficiency** (KPI-03 and KPI-04): offloading state reasoning to lightweight symbolic executors reduces latency by $\sim$65% and token consumption by $\sim$77% relative to the pure LLM planner, as detailed in Tables 5 and 6.

## 5.2 Cognitive and Theoretical Normalization

A core analytical observation arising from this experimental setup is that the architectures yield highly uniform correctness metrics across the board. This uniform distribution is directly attributable to the system design choice where all pipelines share the same parser-generated JSON schema representation. Consequently, the empirical comparison predominantly isolates differences in computational cost and efficiency rather than behavioral variation. While the present technical execution is highly optimized, this pattern tends to structurally normalize Soar and ACT-R into functioning as relatively similar execution backends within the Python middleware. This underscores a theoretical constraint: to fully differentiate these architectures, evaluations must look beyond top-level performance boundaries and investigate how each architecture represents, schedules, and resolves symbolic queries internally.

## 5.3 Limitations and Assumptions

The following constraints affect the overarching hybrid framework rather than isolated pipelines.

The ACT-R branch relies on a custom Python implementation (`Pipelines/ACTR/model.py`) rather than the canonical Lisp ACT-R runtime or `pyactr`, as described in Section 3.5. While core cognitive bottlenecks are preserved, full subsymbolic activation equations and canonical conflict-resolution internals are absent.

The static 4x4 grid environment (Section 3.1) deliberately omits sensor noise, partial observability, and continuous motion kinematics. This optimally isolates text-to-action reasoning but under-represents the uncertainties and temporal dynamics inherent to physical robotic deployment.

Finally, the pure LLM baseline acts as a simulated executor. While it is strictly prompt-constrained to emit standard execution traces for comparability, any observed failures reflect the probabilistic nature of prompt-governed reasoning rather than true algorithmic state transitions. It operates under the assumption that the OWL model serves as a correct and complete world oracle, purposefully prioritizing grounded execution guarantees over broad, unconstrained linguistic expressiveness.

# 6 Conclusion and Future Work

The symbolic frameworks (Soar, ACT-R, and Ontology) consistently outperformed the pure LLM baseline on efficiency while all four pipelines achieved identical correctness scores, validating the hybrid LLM-to-symbolic pattern as the preferred architecture for grounded robotic instruction following.

Future work could expand upon these structural findings along three main axes. First, to deepen the cognitive and theoretical comparison, experimental branches could explore the distinct cognitive differences between these architectures by lifting the uniform JSON parser constraints and testing unique procedural mechanisms native to each engine.

Second, from an interface perspective, the framework would benefit from a live React or Streamlit telemetry dashboard. Such a tool could visually capture the 4x4 grid, tracking dynamic object positions, agent state trajectories, and trace execution events step-by-step for each isolated pipeline. While valuable for real-time analysis, the design and implementation of this dedicated visualization layer remained beyond the scope of this initial study.

Finally, a promising direction for bridging abstract reasoning with physical simulation involves a minimal ROS 2 integration layer. This integration would allow the validated JSON action schemas to be published directly to simulated robotic platforms, such as a TurtleBot3 operating within a Gazebo environment. Exploring this avenue would provide a valuable opportunity to evaluate the resilience of hybrid text-to-action control under realistic physics, spatial noise, and execution constraints, serving as a logical next step for validating these architectures in dynamic environments.

# References


[1] J. E. Laird, A. Newell, and P. S. Rosenbloom, "Soar: An architecture for general intelligence," *Artificial intelligence*, vol. 33, no. 1, pp. 1–64, 1987.

[2] J. E. Laird, *The Soar cognitive architecture*. MIT press, 2019.

[3] J. E. Laird, E. S. Yager, M. Hucka, and C. M. Tuck, "Robo-soar: An integration of external interaction, planning, and learning using soar," *Robotics and Autonomous Systems*, vol. 8, no. 1-2, pp. 113–129, 1991.

[4] J. E. Laird, K. R. Kinkade, S. Mohan, and J. Z. Xu, "Cognitive robotics using the soar cognitive architecture." in *CogRob@ AAAI*, 2012.

[5] J. R. Anderson, D. Bothell, M. D. Byrne, S. Douglass, C. Lebiere, and Y. Qin, "An integrated theory of the mind." *Psychological review*, vol. 111, no. 4, p. 1036, 2004.

[6] R. Sun, *Cognition and multi-agent interaction: From cognitive modeling to social simulation*. Cambridge University Press, 2006.

[7] F. E. Ritter, F. Tehranchi, and J. D. Oury, "Act-r: A cognitive architecture for modeling cognition," *Wiley Interdisciplinary Reviews: Cognitive Science*, vol. 10, no. 3, p. e1488, 2019.

[8] J. G. Trafton, L. M. Hiatt, A. M. Harrison, I. Tamborello, P. Franklin, S. S. Khemlani, and A. C. Schultz, "Act-r/e: An embodied cognitive architecture for human-robot interaction," *Journal of Human-Robot Interaction*, 2013.

[9] W. G. Kennedy, M. D. Bugajska, M. Marge, W. Adams, B. R. Fransen, D. Perzanowski, A. C. Schultz, and J. G. Trafton, "Spatial representation and reasoning for human-robot collaboration," in *AAAI*, vol. 7, 2007, pp. 1554–1559.

[10] U. Kurup and C. Lebiere, "What can cognitive architectures do for robotics?" *Biologically Inspired Cognitive Architectures*, vol. 2, pp. 88–99, 2012.

[11] E. Prestes, J. L. Carbonera, S. R. Fiorini, V. A. Jorge, M. Abel, R. Madhavan, A. Locoro, P. Goncalves, M. E. Barreto, M. Habib *et al.*, "Towards a core ontology for robotics and automation," *Robotics and Autonomous Systems*, vol. 61, no. 11, pp. 1193–1204, 2013.

[12] T. Haidegger, M. Barreto, P. Gonçalves, M. K. Habib, S. K. V. Ragavan, H. Li, A. Vaccarella, R. Perrone, and E. Prestes, "Applied ontologies and standards for service robots," *Robotics and autonomous systems*, vol. 61, no. 11, pp. 1215–1223, 2013.

[13] J. I. Olszewska, M. Barreto, J. Bermejo-Alonso, J. Carbonera, A. Chibani, S. Fiorini, P. Goncalves, M. Habib, A. Khamis, A. Olivares *et al.*, "Ontology for autonomous robotics," in *2017 26th IEEE international symposium on robot and human interactive communication (RO-MAN)*. IEEE, 2017, pp. 189–194.

[14] C. Schlenoff, E. Prestes, R. Madhavan, P. Goncalves, H. Li, S. Balakirsky, T. Kramer, and E. Miguelanez, "An ieee standard ontology for robotics and automation," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 1337–1342.

[15] M. Tenorth and M. Beetz, "Knowrob—knowledge processing for autonomous personal robots," in *2009 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2009, pp. 4261–4266.

[16] A. Olivares-Alarcos, D. Beßler, A. Khamis, P. Goncalves, M. K. Habib, J. Bermejo-Alonso, M. Barreto, M. Diab, J. Rosell, J. Quintas *et al.*, "A review and comparison of ontology-based approaches to robot autonomy," *The Knowledge Engineering Review*, vol. 34, p. e29, 2019.

[17] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[19] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[20] H. Jeong, H. Lee, C. Kim, and S. Shin, "A survey of robot intelligence with large language models," *Applied sciences*, vol. 14, no. 19, p. 8868, 2024.

[21] A. Lykov and D. Tsetserukou, "Llm-brain: Ai-driven fast generation of robot behaviour tree based on large language model," in *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*. IEEE, 2024, pp. 392–397.

[22] Z. Yang, S. S. Raman, A. Shah, and S. Tellex, "Plug in the safety chip: Enforcing constraints for llm-driven robot agents," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 435–14 442.

[23] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PmLR, 2021, pp. 8748–8763.

[24] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.